

```
// Name: Scott Brandon Finkelstein
// Course: CS 201 - Data Structures and Algorithms
// Description: This program allows a user to build, insert, delete, and
traverse a BST
```

```
import java.util.Scanner;
```

```
public class Main {
    // Class Node with the data and the child nodes
    class Node {
        int data;
        Node leftChild;
        Node rightChild;

        Node(int data) {
            this.data = data;
            leftChild = rightChild = null;
        }
    }

    // root node for the binary tree
    Node root;

    // sets root equal to null, clearing the tree
    public void clear() {
        root = null;
    }

    // Constructor method
    public Main() {
        root = null;
    }

    // Insert method for new values in the tree
    public void insert(int key) {
        root = insertNode(root, key);
    }

    // Insert recursive call for inserting from the root, in the right
place
    public Node insertNode(Node node, int key) {
        if (node == null) {
            node = new Node(key);
            return node;
        }
        if (key <= node.data) {
            node.leftChild = insertNode(node.leftChild, key);
        } else if (key > node.data) {
            node.rightChild = insertNode(node.rightChild, key);
        }
        return node;
    }

    // Find method asking for the node to find a search key
```

```

public Node find(int key) {
    Node node = findNode(root, key);
    return node;
}

// Find recursive method using the root node.
public Node findNode(Node node, int key) {
    // if the search key is found at this node, then return the node
    if (key == node.data) {
        return node;
    }
    // if the key is less than the node we are viewing, go left down
the search tree (i.e., go
// to node.leftChild)
    if (key <= node.data) {
        // if the left child node (node.leftChild) is null, then
return null
        if (node.leftChild == null) {
            return null;
        } else { // otherwise, if its value is not null, return the
next left child node
            return findNode(node.leftChild, key);
        }
    }
    // if the search key is > the node we are looking at, go right
down the search tree
    else if (key > node.data) {
        // if the right child node is null, return null
        if (node.rightChild == null) {
            return null;
        } else { // otherwise, if not null, return the right child
node
            return findNode(node.rightChild, key);
        }
    }
    return node;
}

// builds balanced preset BST
public void buildPreset() {
    clear();
    int[] data = {4, 2, 6, 1, 3, 5, 7};
    for (int i = 0; i < data.length; i++)
        insert(data[i]);
}

static Node getSuccessor(Node curr) {
    curr = curr.rightChild;
    while (curr != null && curr.leftChild != null) {
        curr = curr.leftChild;
    }
    return curr;
}

```

```

public void deleteNode(int x) {
    root = deleteNode(root, x);
}

// Delete a node with value, x, from the BST
static Node deleteNode(Node root, int x) {
    if (root == null)
        return root;
    if (root.data > x) {
        root.leftChild = deleteNode(root.leftChild, x);
    } else if (root.data < x) {
        root.rightChild = deleteNode(root.rightChild, x);
    } else {
        // node with 0 or 1 child
        if (root.leftChild == null)
            return root.rightChild;
        if (root.rightChild == null)
            return root.leftChild;

        // Node with 2 children
        Node succ = getSuccessor(root);
        root.data = succ.data;
        root.rightChild = deleteNode(root.rightChild, succ.data);
    }
    return root;
}

// interfaces with the true printInOrder method below
public void printInOrder() {
    printInOrder(root);
}

// prints an in-order traversal of the BST
public void printInOrder(Node node) {
    if (node == null) {
        return;
    }
    // traverse left subtree
    printInOrder(node.leftChild);
    // visit node
    System.out.print(node.data + " ");
    // traverse right subtree
    printInOrder(node.rightChild);
}

// interfaces with the true printPreOrder method below
public void printPreOrder() {
    printPreOrder(root);
}

// prints a pre-order traversal of the BST
public void printPreOrder(Node node) {
    if (node == null) {
        return;
    }

```

```

    }
    // visit node
    System.out.print(node.data + " ");
    // traverse left subtree
    printPreOrder(node.leftChild);
    // traverse right subtree
    printPreOrder(node.rightChild);
}

// interfaces with the printPostOrder method below
public void printPostOrder() {
    printPostOrder(root);
}

// prints a post-order traversal of the BST
public void printPostOrder(Node node) {
    if (node == null) {
        return;
    }
    // traverse left subtree
    printPostOrder(node.leftChild);
    // traverse right subtree
    printPostOrder(node.rightChild);
    // visit node
    System.out.print(node.data + " ");
}

// method to print the menu options for the user interface
private static void printMenu() {
    System.out.println("\n===== BST MENU =====");
    System.out.println("1) Create a preset binary search tree with
nodes 1 through 7");
    System.out.println("2) Add a node");
    System.out.println("3) Delete a node");
    System.out.println("4) Print nodes by InOrder");
    System.out.println("5) Print nodes by PreOrder");
    System.out.println("6) Print nodes by PostOrder");
    System.out.println("7) Exit program");
    System.out.println("=====");
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    Main binaryTree = new Main();
    // program loops while running = true
    boolean running = true;

    while (running) {
        printMenu(); // show menu options 1-7
        int userInput = scanner.nextInt();

        if (userInput == 1) {
            binaryTree.buildPreset();
        }
    }
}

```

```

        else if (userInput == 2) {
            System.out.print("Enter a number to insert into the tree:");
            binaryTree.insert(scanner.nextInt());
        }

        else if (userInput == 3) {
            System.out.print("Enter a number to remove from the tree:");
            int val = scanner.nextInt();
            binaryTree.deleteNode(val);
        }

        else if (userInput == 4) {
            binaryTree.printInOrder();
        }

        else if (userInput == 5) {
            binaryTree.printPreOrder();
        }

        else if (userInput == 6) {
            binaryTree.printPostOrder();
        }

        else if (userInput == 7) {
            System.out.println("Exiting program...");
            running = false;
        } else {
            System.out.print("Invalid input - try again!");
        }
    }
    scanner.close();
}

```

```

// sources used:
// https://www.geeksforgeeks.org/dsa/binary-search-tree-traversal-inorder-preorder-post-order/
// https://study.com/academy/lesson/practical-application-for-data-structures-search-trees.html
// https://study.com/academy/lesson/binary-trees-applications-implementation.html
// https://study.com/academy/lesson/binary-search-trees-definition-uses.html
// https://www.geeksforgeeks.org/dsa/deletion-in-binary-search-tree/

```